

Market Foundry

From documents to intelligence — built your way.

Matthew Wong **Jessica Batbayar** **Marija Vukic** **Biwei Huang**
maw034@ucsd.edu jbatbayar@ucsd.edu mvukic@ucsd.edu bih007@ucsd.edu

Abstract

This work presents Market Foundry, a production-oriented pipeline for transforming unstructured financial text into connected, domain-grounded knowledge graphs. Modern financial workflows generate fragmented outputs—summaries, sentiment scores, and event tags—without preserving relationships across filings, earnings calls, and news, limiting their usefulness for downstream analytics. Market Foundry addresses this gap through a document-type-aware, schema-guided, LLM-driven framework that performs document classification, dynamic topic sectioning, and schema-aligned YAML template generation to enable content-adaptive extraction across heterogeneous financial documents. Built on OneKE, the pipeline produces structured entity, event, and relationship triples integrated into Neo4j, forming an intelligence layer that supports analytics, RAG, and causal inference workflows. The full pipeline is also exposed as a hosted REST API, allowing users to submit documents and receive structured triples without local setup or GPU infrastructure. Market Foundry demonstrates how schema-guided LLM extraction can support scalable financial knowledge graph construction that captures how real-world financial events evolve across documents, time, and market actors.

Keywords: Large Language Models, Knowledge Graph Construction, Financial Document Intelligence, Schema-Guided Extraction, Causal Analysis, Information Extraction, Retrieval-Augmented Generation, Financial NLP, REST API, Cloud Deployment

Website: <https://jessicabat.github.io/market-foundry/>

Code: <https://github.com/jessicabat/market-foundry>

API Documentation:

<https://marija-vukic--market-foundry-api-fastapi-app.modal.run/docs#/>

Table of Contents

1	Introduction	3
2	Methods	6
3	Results	11
4	Discussion	17
5	Conclusion	23
	References	24
	Appendices	A1
A	Additional Figures	A1

1 Introduction

1.1 Overview

Unstructured financial text, including regulatory filings, earnings call transcripts, news articles, and research reports, contains critical context about real-world market events and decision drivers. This information is typically processed in isolation through summarization, sentiment scoring, or event tagging, leaving relationships between entities, events, and financial outcomes fragmented across documents and inaccessible to analytical systems.

Existing approaches fail to address this gap in two key ways. General-purpose LLMs can summarize individual documents but cannot reliably preserve structured relationships across a corpus, and are prone to hallucination when asked to synthesize facts across multiple sources [Ganesh et al. \(2024\)](#). Rule-based extraction systems, on the other hand, require rigid ontologies that cannot adapt to the heterogeneous structure and terminology of real financial documents. Neither approach produces a grounded, queryable representation of the relationships that drive financial reasoning.

Knowledge graphs address this by organizing information into entities, events, and relationships, enabling structured reasoning over complex, multi-source financial narratives [Arun et al. \(2025\)](#). Rather than asking a model to recall facts, a knowledge graph grounds every assertion in an explicitly extracted, document-traceable triple, making the output verifiable, queryable, and resistant to fabrication.

In this work, we introduce Market Foundry, a document-type-aware, schema-guided pipeline for constructing financial knowledge graphs from heterogeneous financial documents. The system is designed to be accessible without local GPU infrastructure, and exposes its full pipeline as a hosted REST API.

1.2 Literature Review

1.2.1 OneKE

OneKE is a schema-guided knowledge extraction system that is compatible with both Anaconda and Docker environments, providing flexibility for local development as well as containerized deployment. The Dockerized design ensures reproducibility and scalability, while the Anaconda-based setup allows for lightweight experimentation and environment management across diverse computing setups.

It supports the extraction of knowledge ranging from web pages, to raw PDF books, and other text formats for domains such as science, news, etc. OneKE operates effectively through three core agents that compose its autonomous flow: the Schema Agent, Extraction Agent, and Reflection Agent (Figure 1). These three agents respectively handle schema analysis of the various data types, knowledge extraction via large language models, and the debugging and handling of errors throughout the extraction process [Luo et al. \(2025\)](#). In our project, OneKE serves as a foundation for constructing structured knowledge graphs

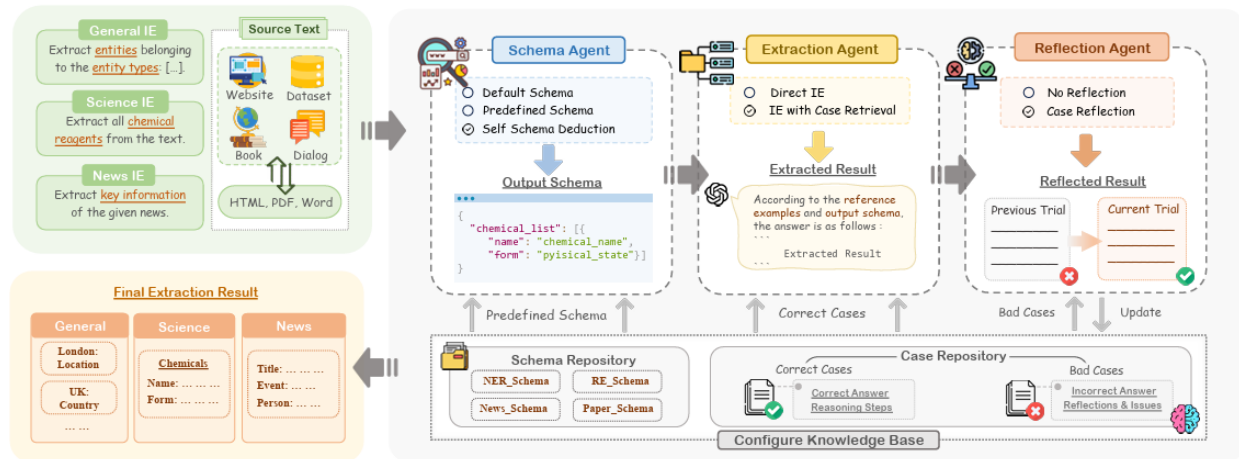


Figure 1: OneKE system overview for multi-domain knowledge extraction.

from unstructured text data. Its schema-guided triple extraction based on the (Subject, Relation, Object) format enables consistent relational representation across data sources. The resulting structured representation establishes a foundation for causal structure learning, where the intricate relationships identified by OneKE can be systematically leveraged to improve causal inference discovery.

1.2.2 Financial Knowledge Graph Construction at Scale

Prior work has begun exploring automated construction of financial knowledge graphs from regulatory documents. Arun et al. (2025) introduce FinReflectKG, a research pipeline and open-source dataset of knowledge graph triples constructed from the 2024 SEC 10-K filings of all S&P 100 companies, publicly released on HuggingFace. Their methodology comprises four stages: intelligent document parsing via docling, a table-aware semantic chunker that preserves financial tables as atomic 2048-token chunks, schema-guided iterative triple extraction in three configurable modes (single-pass, multi-pass, and reflection-agent-based), and a holistic evaluation framework combining rule-based CheckRules compliance, coverage ratios, Shannon entropy, and LLM-as-a-Judge scoring. The schema consists of ten entity types and ten relationship types, defined once through an AI-assisted process in which an LLM proposes candidate schemas that are reviewed and approved by financial subject-matter experts. The resulting fixed schema is applied uniformly across all documents. Empirical results show that the reflection-agent mode achieves the best balance of quality and coverage, attaining 64.8% full compliance across all CheckRules policies and outperforming single-pass and multi-pass baselines on precision, comprehensiveness, and relevance.

FinReflectKG validates the core idea that schema-guided LLM extraction can produce high-quality financial knowledge graphs from regulatory text, but its design is scoped to producing a one-time research dataset from a single document type rather than serving as a generalizable extraction pipeline. The schema is permanently fixed to SEC 10-K structure,

cross-document co-reference resolution is explicitly left as future work, and the system produces static triple outputs rather than integrating into a live queryable graph. The authors themselves identify schema-free construction across diverse document types as an open research direction. Market Foundry is designed to address these gaps as a deployable pipeline rather than a dataset release. It automatically generates tailored extraction schemas per document without any manual ontology design, supports five distinct financial document types out of the box, and integrates extracted triples directly into a live Neo4j graph database enabling structured querying and relationship tracing across the full document corpus. This makes Market Foundry applicable to the real-world financial workflows where document variety and continuous ingestion are the norm, not the exception.

1.2.3 Schema-Guided Information Extraction

Schema-guided extraction constrains LLM outputs to predefined structural templates, substantially improving reliability. [Bai et al. \(2024\)](#) introduced schema-driven IE across heterogeneous sources, demonstrating that human-authored templates enable consistent structured extraction from both tabular and textual inputs. [Shrimal et al. \(2025\)](#) advanced this further with PARSE, an automated schema optimization framework showing up to 64.7% accuracy gains over unconstrained extraction, alongside SCOPE, a reflection-based guardrail for schema compliance enforcement.

1.2.4 Large Language Model Hallucination

Large Language Model (LLM) hallucination refers to the generation of syntactically fluent and plausible text. It presents a critical challenge in high-stakes domains such as finance, where errors in extracted facts can propagate directly into analytical outputs. This risk is compounded in multi-document settings: recent work shows that in multi-document summarization tasks, up to 75% of generated content may contain hallucinated elements, often appearing as fabricated details or "ungrounded insights" that lack basis in the source text [Belem et al. \(2025\)](#). In financial document analysis specifically, [Shah et al. \(2025\)](#) demonstrate that state-of-the-art models including GPT-4o and Llama-3 hallucinate financial information even for well-documented public companies, with hallucination rates paradoxically increasing for larger firms. For knowledge graph construction pipelines, these errors manifest as hallucinated triples or non-existent entities that undermine the structural integrity of the resulting graph.

Schema- and ontology-constrained extraction provides a structural mitigation strategy. Grounding retrieval and generation in verified structured representations has been shown to significantly reduce hallucination rates compared to standard LLM baselines [Yu and McQuade \(2025\)](#), and ontology-grounded knowledge graph construction further reduces downstream hallucination in question-answering tasks relative to unconstrained generation [Ali, Taha and Morsey \(2026\)](#). These findings directly motivate Market Foundry's schema-guided, graph-first design.

1.3 Contributions

This work makes the following contributions:

1. We propose **Market Foundry**, a document-type-aware pipeline that combines ensemble document classification, dynamic topic-driven schema generation, and schema-guided triple extraction built on the OneKE framework to construct financial knowledge graphs from heterogeneous textual sources.
2. We introduce **dynamic schema generation via semantic topic routing**, which automatically produces document-specific YAML extraction configurations tailored to each document’s predicted class and extracted themes, enabling adaptive, multi-perspective extraction across diverse financial document types.
3. We present a **production-oriented REST API deployment** via Modal, enabling knowledge graph construction from financial documents without any local GPU infrastructure.
4. We provide **empirical evaluation** of the full pipeline across document classification accuracy, YAML generation robustness, extraction coverage, graph connectivity, and API runtime performance on a curated corpus of real financial documents.

2 Methods

2.1 Data Acquisition

Our goal was not to build a large-scale crawler, but to assemble a small, well-controlled corpus of financial text that we could use to prototype and debug schema-guided extraction with OneKE. All documents were collected manually and stored as raw PDF files under the `Papers/` directory.

2.1.1 Selection criteria

We applied two simple but strict criteria when building this corpus:

- **Domain relevance:** documents must be clearly within the finance or financial-technology domain (i.e., equity research, asset management, earnings, or market news).
- **Language:** documents must be written in English to match the capabilities of the LLMs used in OneKE.

Within these constraints, we deliberately chose documents that vary along two dimensions: (i) source type and (ii) length/complexity. This allows us to test whether the same extraction framework can handle both long, technical reports and short news articles.

2.1.2 Corpus composition

The resulting pilot corpus consists of PDFs drawn from:

- **Academic and practitioner-oriented research** (ex: a 290-page quantitative finance primer and a CFA Institute piece on the evolution of quantitative asset management).
- **Equity research and reports** (ex: a 36-page ASML initiation of coverage report).
- **Methodology papers and project reports** (a recent paper on financial statement analysis with large language models and a course project report on financial ML).
- **News articles** (ex: short market news around specific company and crypto events).
- **Earnings call transcripts** (ex: a multi-page sample earnings call used to approximate the long-form conversational text we ultimately want to convert into event-level knowledge graphs).

These documents span from 2 to 290 pages and cover publication dates from 2008 to 2025. This diversity allows us to observe how OneKE behaves under different document styles, densities of numerical information, and levels of financial jargon, while keeping the total corpus small enough for rapid iteration.

2.1.3 Organization

Each manually collected PDF is stored in the `Papers/` directory using descriptive file names. The organization allows for easy updates to files as well as further additions to the corpus.

2.2 Setup

Market Foundry can be run in two ways: via the hosted REST API or through a local installation.

2.2.1 API

The hosted API requires no local setup. Users can submit documents and retrieve structured triples directly over HTTP without managing dependencies or GPU infrastructure. The pipeline follows a three-step async pattern:

1. **Submit a document** and receive a `job_id`:

```
curl -X POST "https://marija-vukic--market-foundry-api-fastapi-app\
.modal.run/process" \
  -F "file=@/path/to/your/document.pdf" \
  -F "neo4j_uri=neo4j+s://xxxx.databases.neo4j.io" \
  -F "neo4j_username=neo4j" \
  -F "neo4j_password=yourpassword"
```

2. **Poll for results** using the returned `job_id`:

```
curl "https://marija-vukic--market-foundry-api-fastapi-app\
.modal.run/result/fc-JobID"
```

3. Query your graph via the API (optional):

```
curl -X POST "https://marija-vukic--market-foundry-api-fastapi-app\
.modal.run/query" \
-F "cypher=MATCH (s)-[r]->(o) RETURN s.name, type(r), o.name \
LIMIT 25" \
-F "neo4j_uri=neo4j+s://xxxx.databases.neo4j.io" \
-F "neo4j_username=neo4j" \
-F "neo4j_password=yourpassword"
```

Full API documentation and usage examples are available at <https://marija-vukic--market-foundry-api-fastapi-app.modal.run/docs#/>.

2.2.2 Local Installation

To reproduce the full pipeline locally, clone the repository and optionally ensure Docker Desktop is installed. To customize the extraction, open `extraction_config.yaml` in the `src/utils` folder. For graph construction, provision a Neo4j instance either locally via Docker or remotely through Neo4j AuraDB, and provide the corresponding connection URL and credentials in the configuration.

Market Foundry supports two local environment options. To set up a Conda environment:

```
conda env create -f environment.yml
conda activate <environment_name>
```

To set up with Docker using the Dockerfile (include `-gpus all` only if a GPU is available):

```
docker build -t market-foundry .
docker run --gpus all -v $(pwd):/app market-foundry
```

2.3 Processing Pipeline

The following includes the processes we took to create our pipeline. The sections include: Document Classification, Dynamic Topic Extraction, and Knowledge Extraction. Figure 2 provides an overview of the end-to-end flow.

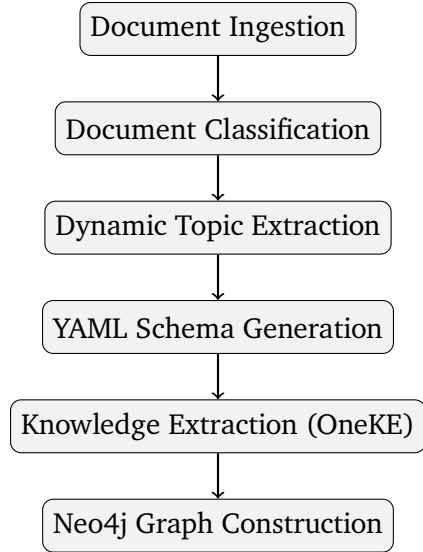


Figure 2: Market Foundry end-to-end processing pipeline.

2.3.1 Document Classification

We support internal uploads in multiple file formats and normalize them into a common text representation before classification, including PDFs, Word documents, HTML pages, JSON exports, and plain text files. The document classification module predicts over the same set of financial document types used throughout the pipeline: earnings call transcripts, press releases, research papers, SEC filings, and news articles.

Our document classifier uses a stacked ensemble architecture composed of three stages. The first stage is a TF-IDF-based multinomial logistic regression model that produces class probabilities over the five supported document types. In parallel, we run a semantic k-nearest neighbors classifier over a reference corpus of prototypical documents using sentence-transformer embeddings. To keep semantic retrieval informative while controlling input length, the KNN component embeds a managed document snippet that preserves leading content, trailing boilerplate, and a bounded portion of the document body.

The outputs of these two base classifiers are combined by a calibrated stacking model. The stacker takes as input the TF-IDF class probabilities, the KNN per-class similarity scores, confidence margins from both models, and a small set of structural cue features designed to capture patterns common in corporate communications. These include lexical and formatting cues associated with press releases, such as wire-service markers, investor relations contact blocks, newsroom language, and announcement-style phrasing. The stacker produces a refined multiclass probability distribution over the full document taxonomy.

To improve performance on the most difficult classification boundary, the pipeline includes a targeted second-stage binary classifier for press releases and news articles. When the stacked model identifies a document as plausibly belonging to either of these two classes, a specialized reranker is applied to re-evaluate only that boundary using boundary-specific lexical and structural features. The final document label is then passed to downstream

sectioning, topic extraction, and schema-guided knowledge extraction.

2.3.2 Dynamic Topic Extraction

After document classification, we perform dynamic topic extraction using a two-stage prompt-engineered semantic routing process built on either the Qwen2.5-1.5B-Instruct model or a model specified through the extraction configuration via an API endpoint. Rather than relying on fixed section headers or rule-based segmentation, we treat sectioning as a semantic interpretation task, consistent with recent work showing that dynamic schema routing improves extraction coverage over fixed templates [Bai et al. \(2024\)](#).

In the first stage, the model scans the normalized document and produces a compact set of reusable semantic topics representing its primary themes (e.g., capital allocation, infrastructure investment, risk disclosures, operational performance). The prompt enforces structured JSON output and constrains topic scope to ensure downstream schema compatibility. These topics serve as semantic proxies for latent document structure.

In the second stage, we generate three distinct YAML extraction configurations per document using the shared topic set and predicted document class. Each configuration emphasizes a different extraction perspective (e.g., financial metrics, corporate relationships, or technological dependencies), specifying tailored entity and relation constraints. These configurations are executed independently, and their extracted triples are aggregated into the final graph. This multi-configuration strategy increases relational coverage and reduces under-extraction from any single schema view.

If dynamic topic sectioning or YAML generation fails, the pipeline defaults to the predefined configuration associated with the document’s predicted class, ensuring consistent processing across the corpus.

2.3.3 Knowledge Extraction

Once a document has passed through classification and topic extraction, it is forwarded to the knowledge extraction stage. At this point, the pipeline has already determined the document type and generated a topic-aware YAML configuration that specifies the extraction schema, model settings, and input text. This configuration is then passed to the OneKE framework, which performs schema-guided information extraction over the document content.

The output of this stage is a structured collection of triples, where each record contains a head entity, relation, and tail entity, along with associated type information. In practice, triples are represented as dictionaries with fields such as `head`, `head_type`, `relation`, `relation_type`, `tail`, and `tail_type`. This format provides a normalized representation that can be used both for inspection in JSON form and for downstream graph construction.

After extraction, the validated triples are written to Neo4j for knowledge graph construction. Each triple is converted into graph structure by mapping the head and tail fields to

nodes and the relation field to an edge between them, while preserving document meta-data and type information where available. This allows extracted knowledge to move from unstructured source text into a queryable graph representation that supports downstream exploration, retrieval, and analysis.

3 Results

While the classification models were evaluated on a larger labeled dataset of 71 documents, the end-to-end extraction pipeline was tested on a smaller manually curated corpus of 15 financial documents used for graph construction experiments. YAML generation statistics were collected from a broader stress test involving 112 documents.

3.1 Document Classification Statistics

To assess the dual-model classification architecture, we tested the pipeline against a curated dataset comprising 71 financial documents from five target categories. The system successfully extracted features, reconciled conflicting model predictions, and directed each file to its appropriate downstream processing queue.

The ensemble’s aggregate performance yielded the following results:

- Total evaluated documents: 71
- Overall Accuracy: 78.9
- Macro F1 Score: 0.802
- Weighted F1 Score: 0.781

These metrics demonstrate robust overall prediction capabilities. However, the gap between the macro and weighted F1 scores indicates that predictive accuracy fluctuates depending on class size and the underlying data distribution.

3.1.1 Class-Level Performance

The ensemble model excelled at identifying document types characterized by rigid formatting and highly specific terminology. In particular, the pipeline produced excellent F1 scores for Earnings Calls (0.923), Research Papers (0.933), and SEC Filings (0.973). By combining TF-IDF feature extraction with k-nearest neighbors semantic embeddings, the system reliably isolates these well-defined financial formats.

We evaluate the balance between precision and recall using the standard F1 score, calculated as:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Alongside precision and recall, we report the support for each category, which denotes the true number of document instances present in the evaluation corpus. This metric provides

essential context for the statistical reliability of the class-level scores and serves as the proportional weighting factor for the aggregate weighted F1 calculation.

Within the Market Foundry architecture, recall serves as the most critical classification metric. Because the document classifier acts as the primary routing engine for the downstream extraction phase, a category’s recall score directly dictates the proportion of documents that are assigned the correct schema. Prioritizing high recall ensures that critical financial data is not misrouted to an incompatible extraction pipeline, which could otherwise lead to omitted fields, hallucinated data, or system fallbacks.

The evaluation yielded the following detailed metrics per category:

Document Type	Precision	Recall	Support
Earnings Call	0.923	0.923	13
News Article	0.500	0.917	12
Press Release	0.800	0.400	20
Research Paper	0.875	1.000	7
SEC Filing	1.000	0.947	19

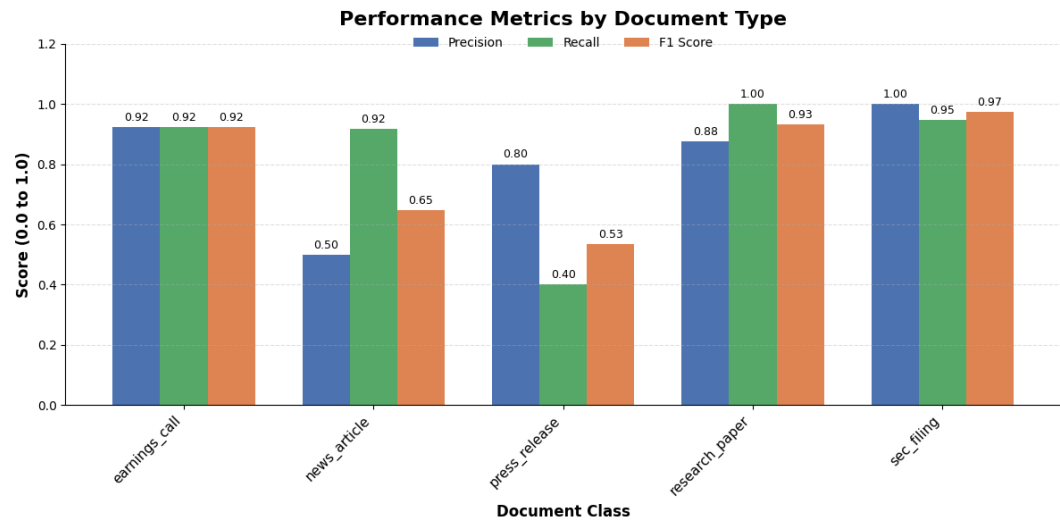


Figure 3: Class-level performance metrics comparing Precision, Recall, and F1 Score across the five evaluated document categories.

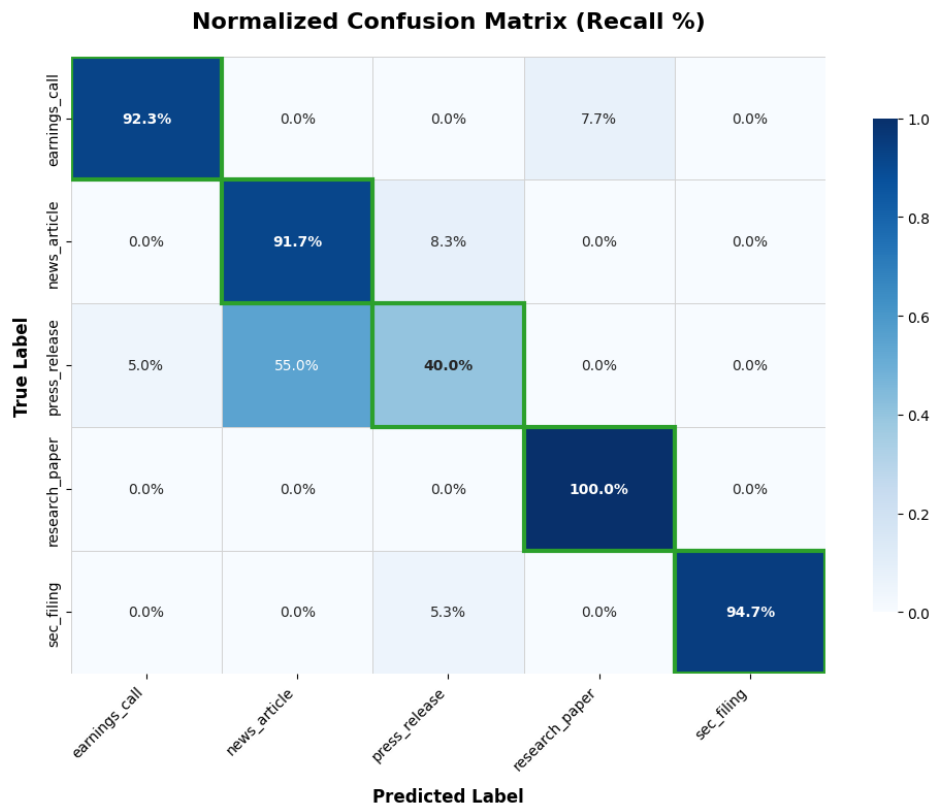


Figure 4: Normalized confusion matrix displaying recall percentages. The matrix highlights the strong predictive accuracy for core financial documents alongside the specific routing leakage of press releases into the news article category.

3.1.2 Semantic Overlap and Misclassification

Testing revealed a significant boundary issue between the `news_article` and `press_release` categories. Although the system maintained satisfactory precision (0.800) when assigning the Press Release label, its recall was notably weak (0.400). A review of the confusion matrix shows that 12 out of 20 actual press releases were incorrectly categorized as news articles.

This misclassification highlights an inherent challenge when applying semantic vector models to corporate media. Press releases are specifically drafted to emulate the objective style, vocabulary, and structure of traditional journalism, causing their embeddings to occupy the same vector space as news articles. While the 78.9% baseline accuracy is reliable and grounded in cleanly separated reference data, differentiating between these deeply similar classes in future versions will likely require hard-coded keyword rules, such as scanning for specific wire service datelines, to cleanly override the semantic routing.

3.2 Topic Extraction & YAML File Generation

To evaluate the robustness of the dynamic configuration pipeline, we tracked topic extraction and YAML generation outcomes across the full corpus. For each document, the system attempts to generate three schema-specialized YAML extraction configurations derived from its predicted class and dynamically extracted topics.

Across 112 processed documents, dynamic YAML generation succeeded for 53 documents (47.3%), producing 159 adaptive configuration files. The remaining documents were processed using classification-specific fallback schemas, ensuring full corpus coverage with domain-appropriate extraction constraints. This hybrid approach reflects an intentional design choice: adaptive, multi-perspective extraction is applied where document structure supports it, while a robust fallback guarantees continued processing regardless of document complexity.

When fallback was triggered, failures originated at the topic extraction stage, prior to YAML generation itself. For longer documents, the model exceeded its maximum context window during topic extraction, causing degradation in instruction-following behavior. This context-window saturation produced malformed or incomplete topic outputs, which in turn prevented valid YAML configurations from being generated. The full generation process for all 112 documents took approximately two hours, utilizing the Qwen3-4B-Instruct-2507 model, and running the pipeline locally on an AMD RX 5700 XT GPU.

Fallback was more frequent for long-form regulatory filings, earnings call transcripts, and research papers relative to short-form news articles, consistent with the context-window saturation explanation above. Dialogue-heavy earnings calls introduced additional instability due to interrogative language patterns that resemble instruction prompts, occasionally interfering with strict schema-constrained generation. These findings are consistent with ECT-SKIE [Huang et al. \(2025\)](#), which characterizes earnings call Q&A sections as structurally anomalous relative to other financial document types.

Overall, all 112 documents were successfully processed, with 47.3% benefiting from dynamically generated, document-specific schemas and the remainder handled through classification-aware defaults. This design preserved full corpus coverage while maximizing adaptive extraction where feasible.

3.3 Extraction Output Statistics

We evaluate Market Foundry on the manually curated corpus described in Section 2. Across all processed documents, the system successfully produced structured triple outputs and constructed a unified Neo4j knowledge graph.



Figure 5: An Example Knowledge Graph Consisting of 200 Triples.

Across a 15 document corpus spanning approximately 443 pages, the pipeline generated 790 structured triples, 964 unique entities, and 790 relationships. This yields an average extraction density of approximately 1.78 triples per page across the corpus, demonstrating the pipeline’s ability to compress large volumes of unstructured financial text into compact, structured relational representations suitable for downstream graph construction and querying. To put this in context, each page of financial text yields on average nearly two structured, queryable facts. For example, a single page of an earnings call might produce triples such as (ASML, reports_revenue, \$28B) and (ASML, banned_from_selling, China).

3.3.1 Ontology and Schema Granularity

Rather than mapping to a rigid, predefined ontology, the Market Foundry pipeline dynamically generates a highly granular, open-schema taxonomy tailored to the text. Our evaluation identified 320 distinct node labels and 423 distinct relationship types across 443 pages of heterogeneous financial documents, reflecting the expressive flexibility of the open-schema design. This granularity enables fine-grained, context-specific semantic edges that would be suppressed by a fixed ontology [Arun et al. \(2025\)](#). The distribution of relationship types follows a long-tail pattern: core generic relationships occur most frequently (e.g., financial_change, service, product_composition), while the system generates highly specific, context-rich semantic edges for niche financial facts (e.g., transaction_margin_dollar_growth, monthly_active_accounts_growth). The long-tail distribution reflects the open-schema design’s prioritization of contextual expressiveness over rigid ontology reuse, at the cost of reduced cross-document normalization.

3.3.2 Graph Connectivity and Coverage

To characterize structural properties of the constructed graph, we computed the average node degree:

$$\text{Average Degree} = 2 \times \frac{\# \text{Relationships}}{\# \text{Nodes}} \quad (1)$$

Empirically, we observed:

- Average out-degree: 0.82
- Average in-degree: 0.82
- Average total degree: 1.64

This indicates a sparse, locally connected structure in which most entities participate in only a small number of explicitly stated relationships, consistent with the explicit, fact-declarative nature of financial reporting language. The near-equal average in-degree and out-degree further confirms that the extraction process does not introduce directional bias, preserving the relational patterns present in the source documents.

To evaluate semantic completeness for core financial entities, we formulated a Relationship Coverage Rate metric. We measured the percentage of target entities connected to expected contextual nodes. Specifically, for Company nodes, we evaluated the presence of explicit `financial_result` edges connecting them to `Financial_Metric` nodes. The observed Relationship Coverage Rate of 4.5% reflects the open-schema design’s preference for generating specialized, context-specific relationship paths over standardized canonical patterns. Rather than relying exclusively on a fixed ontology, the system generates tailored entity and relation types prior to extraction, improving contextual alignment across document types at the cost of the schema fragmentation discussed in Section 3.3.1.

3.4 API Runtime Results

To evaluate the performance of the hosted REST API, we tested the pipeline across documents of varying length and complexity. All startup and execution times reported below reflect solely the runtime of the `MarketFoundryPipeline` container on Modal, and do not include network transfer or client-side overhead. For each document, we recorded the average startup time, total execution time, and the number of nodes and relationships written to Neo4j upon completion.

Startup times were consistently fast across all documents, averaging 7–10 seconds regardless of document length, reflecting the efficiency of Modal’s container warm-up and model caching. Execution time scaled roughly with document length, ranging from 5 minutes for a single-page press release to 26 minutes for a 36-page equity research report. The 290-page Quant Primer exceeded the API’s current timeout limit and could not be processed, consistent with the GPU memory and context-window constraints. These results suggest the API is well-suited for documents up to approximately 50 pages, with larger documents requiring either a local deployment or a custom API instance with upgraded GPU hardware.

Table 1: API runtime results by document page count. All times reflect MarketFoundryPipeline execution via Modal only.

Document	Pages	Startup	Execution	Nodes	Relationships
AmericanFinancialGroup Q4 Results	1	10s	5 min	10	8
CFA PQ Quantitative Asset Mgmt	9	10s	15 min	33	29
ADBE Q3FY25 Earnings Script	25	7s	13.5 min	77	65
ASML Initiation of Coverage	36	8s	26 min	130	136
Quant Primer	290	8s	Timed Out		N/A

4 Discussion

4.1 Interpretation of Graph Structure

The low average node degree reflects a fundamental property of financial reporting language: relationships are explicitly stated within individual documents rather than inferred across them, yielding a graph organized around locally dense, document-level clusters rather than a globally connected structure. This sparsity is not evenly distributed, however.

Certain entities (major firms, key financial metrics, and macroeconomic indicators) appear repeatedly across sections and accumulate edges from multiple surrounding nodes, forming local hubs that anchor the graph’s narrative structure and make the graph amenable to entity-centric querying. The near-equal average in-degree and out-degree further confirms that entities participate as both subjects and objects at comparable rates, indicating that the extraction process preserves the relational directionality present in the source documents rather than introducing systematic bias.

4.2 LLM Behavior & Extraction Quality

Schema-guided extraction proved both versatile and stabilizing. By enforcing a predefined structural format, the model was constrained to produce outputs that adhered to the specified JSON schema [Shrimal et al. \(2025\)](#). Through carefully designed system and user prompts, we significantly reduced hallucinated relations and eliminated unintended chain-of-thought content in the final outputs.

Beyond mitigating hallucinations, schema constraints improved structural consistency. Outputs more reliably conformed to the required format, with fewer malformed entries and missing fields. This consistency was essential for downstream graph construction, as formatting errors would otherwise propagate and lead to failures when inserting data into the Neo4j database.

The model performed strongly on explicitly stated relationships but exhibited greater variability when processing forward-looking or conditional language. Ambiguous financial phrasing often resulted in conservative extraction behavior, where the model omitted po-

tentially valid relations rather than risk producing incorrect assertions.

Common failure modes included minor entity boundary inconsistencies and occasional duplication of semantically equivalent triples. Cross-sentence co-reference resolution also posed challenges, particularly when pronouns referred to entities introduced earlier in the document.

Another behavior we observed involved the Reflection Agent modifying relation labels during its correction stage. In several cases, relation types produced during extraction were renamed to semantically similar labels that were not present in the provided schema. For example, relations defined in the YAML configuration were occasionally replaced with alternative phrasings inferred from the surrounding text. While these substitutions were often semantically reasonable, they violated the schema constraints required by the pipeline and contributed to the inconsistencies of relation types in the constructed graph.

This behavior likely arises from the Reflection Agent’s objective of improving semantic clarity or correcting perceived inconsistencies in the extracted triples. As a result, even when schema-guided extraction is used, enforcement of relation naming remains partially dependent on model behavior. Addressing this issue would likely require stronger constraint mechanisms, models beyond our available compute budget, or additional validation layers to ensure that generated relations remain strictly within the allowed schema.

Finally, even when valid triples were identified internally, OneKE’s implementation occasionally returned empty outputs due to edge-case handling within the pipeline. These issues were mitigated by modifying portions of the source code to better account for such cases, improving overall extraction reliability.

4.3 Engineering Design Choices

Market Foundry was developed by extending the OneKE knowledge extraction framework rather than constructing a pipeline from scratch. Consequently, several architectural decisions were inherited from OneKE’s core design, while others reflect targeted modifications introduced to improve robustness, flexibility, and experimental reliability.

Schema-constrained extraction forms the foundation of the system because it is central to OneKE’s architecture. The framework enforces structured outputs through predefined JSON schemas, ensuring compatibility with downstream knowledge graph construction. This design prioritizes structural determinism over generative flexibility, trading the breadth of unconstrained relation generation for reliable, format-consistent outputs that integrate cleanly with the Neo4j graph store and reduce the propagation of malformed entries into the graph.

The use of Neo4j as the graph database is similarly inherited from OneKE’s original design. The framework is built around a labeled property graph model, and its triple-based extraction format aligns naturally with Neo4j’s directed edge representation. This integration simplifies graph ingestion and querying but places responsibility for ontology normalization and schema consolidation at the application level.

To improve adaptability across diverse financial documents, we extended OneKE by introducing dynamic, document-specific schema generation. Rather than relying exclusively on a fixed ontology, the system generates tailored entity and relation types prior to extraction. This enhances contextual alignment and expressiveness but contributes to ontology fragmentation and a long-tail distribution of relationship types across the corpus.

Additional engineering refinements were implemented to strengthen pipeline stability. These included stricter JSON validation, improved handling of edge cases such as empty outputs, filename sanitization for temporary configuration files, and incremental persistence of extraction results. These modifications addressed practical reliability issues encountered during experimentation and improved reproducibility.

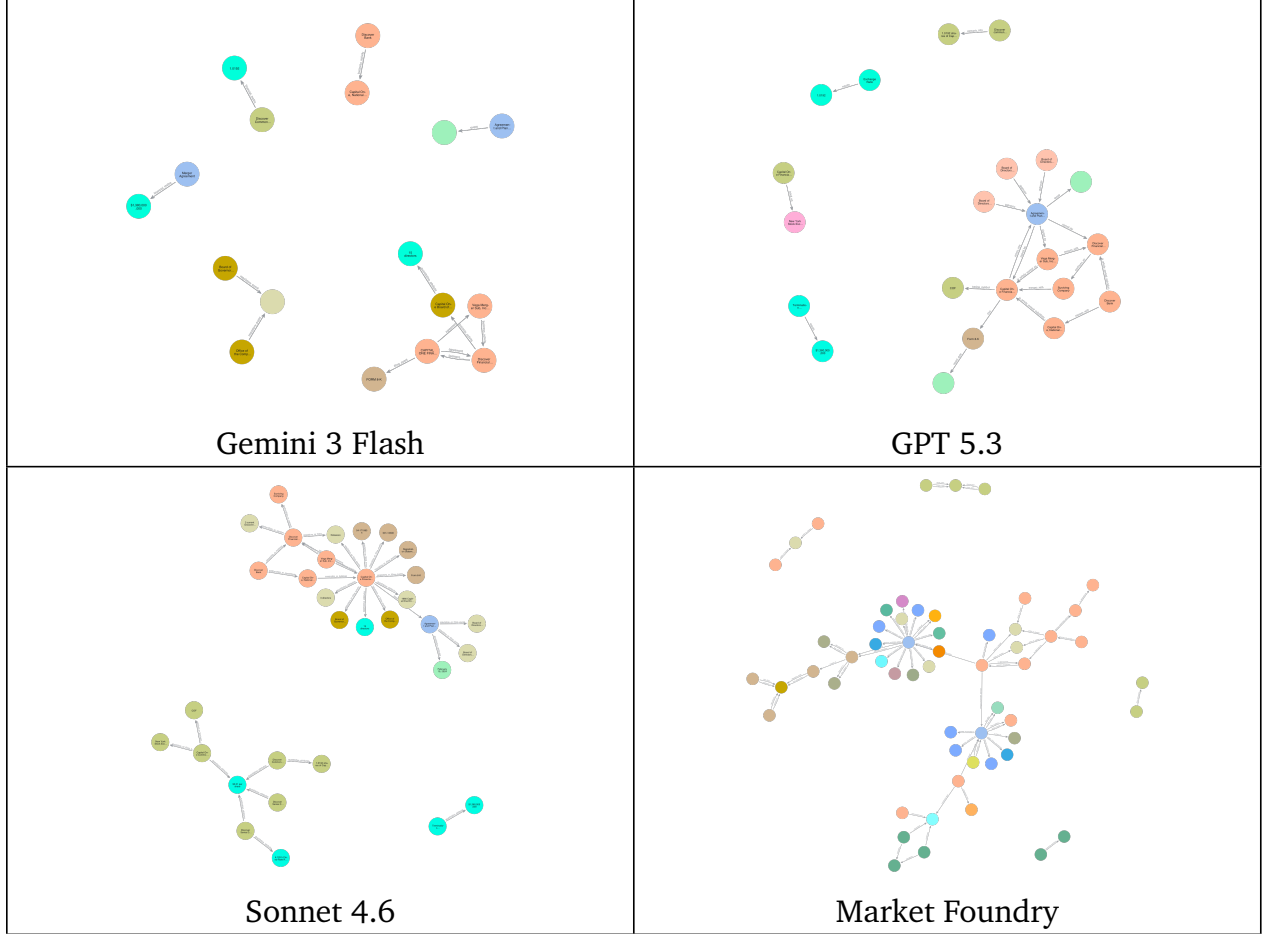
Development relied primarily on local inference to enable rapid iteration and avoid external API costs. While this approach improved experimental control and reduced operational overhead, it limited evaluation to smaller open-source models, which influenced extraction depth, semantic consistency, and coverage behavior.

Overall, Market Foundry represents a layered design approach: leveraging OneKE’s schema-guided extraction and Neo4j-based graph construction backbone, while introducing targeted engineering modifications to enhance robustness, adaptability, and reproducibility within a financial document analysis setting.

4.4 Benchmarking Knowledge Extraction: Market Foundry vs. SOTA Models

We tested several state-of-the-art (SOTA) models on their capabilities for knowledge extraction and JSON-formatted triple output using the same system prompt designed for our pipeline. Specifically, we evaluated Gemini 3 Flash, GPT 5.3, and Sonnet 4.6 against our system. A natural question is why one would not simply use an LLM to summarize the key points of a document instead of using Market Foundry? The answer is structure. LLM summarization can produce organized narrative output, but it lacks the rigid schema required to construct an interpretable knowledge graph suitable for visualization and downstream analysis.

Table 2: Knowledge graphs produced by the test models



4.4.1 Comparative Performance Statistics

Using a 5 page *Capital One 8-K* as the source document, we analyzed the extraction density and relational diversity of each model’s output. Market Foundry, powered by the Qwen3-4b-2507 architecture, produced the highest extraction density despite the significantly larger parameter counts of the frontier models.

Table 3: Extraction volume, schema diversity, and factual accuracy across tested models

Model / Pipeline	Total Triples	Unique Relations	Factual Error Rate
Market Foundry (Qwen3-4b)	68	18	1.47%
Sonnet 4.6	32	6	3.1%
GPT 5.3	22	11	0.0%
Gemini 3 Flash	13	8	7.7%

4.4.2 Key Findings

- High-resolution capture: Market Foundry captured 5.2x more data points than Gemini 3 Flash and 2.1x more than Sonnet 4.6. While larger models tend to produce condensed narrative summaries, the schema-driven architecture captures a higher density of structured information.
- Structural awareness: Market Foundry was the only system to identify and map document-internal metadata, such as linking “Exhibit 2.1” to the primary “Agreement and Plan of Merger.”
- Deep recall: Market Foundry extracted over 20 triples from technical sections (e.g., Forward-Looking Statements and Risk Factors) that were largely ignored by the general-purpose frontier models.
- Scaling potential: These benchmarks were achieved using a lightweight 4B parameter model. Providing Market Foundry with access to larger SOTA models as extraction backends would likely further improve the capture of niche administrative identifiers and complex financial relationships.
- Factual reliability at scale: A triple-level audit against the source document revealed that Market Foundry achieved the lowest factual error rate among the evaluated systems at 1.47% (1 inverted relation direction across 68 triples), lower than Sonnet 4.6 (3.1%) and Gemini 3 Flash (7.7%), and comparable to GPT 5.3 (0% across only 22 triples). Notably, Market Foundry produced zero hallucinated facts; the single error occurred because the relationship between two entities was recorded in the wrong direction for a non-critical administrative triple.

4.5 API vs Running Locally

Market Foundry can be used in two ways: via the hosted REST API or through a local installation. Each mode is suited to different use cases and user priorities.

The hosted API provides immediate access to the full Market Foundry pipeline without requiring any local environment setup, GPU hardware, or dependency management. Users can submit documents and receive structured triples directly over HTTP, making it the preferred option for rapid prototyping, integration into existing workflows, or situations where infrastructure overhead is a constraint. The API is optimized for documents up to approximately 50 pages; very large documents may exceed available GPU memory limits.

Local installation provides greater flexibility and control. Users can swap in any supported model by editing `extraction_config.yaml`, adjust decoding parameters, modify the extraction prompts, and run experiments that require reproducibility over a specific model configuration. This mode is better suited for research workflows, pipeline customization, and evaluation across different model families.

Table 4: Comparison of API and Local deployment modes.

Feature	API	Local
Setup required	None	GPU + dependencies
Model flexibility	Fixed (Qwen 3B)	Any supported model
Custom prompts	No	Yes
Neo4j integration	Optional (BYOD)	Optional (BYOD)
Document size limit	~50 pages	Hardware-dependent
Deployment cost	Free (shared GPU)	Local compute
Best for	Quick extraction	Research & customization

4.6 Limitations

OneKE’s schema-guided knowledge extraction system provided the backbone of our project structure. However, several limitations constrained both the evaluation and practical scalability of Market Foundry.

First, computational requirements limited experimental flexibility. Running open-source transformer models through the Hugging Face Transformers library required substantial memory and storage, even for relatively small 1–4 billion parameter models. These constraints restricted systematic benchmarking across multiple model sizes and decoding configurations. We partially mitigated this limitation by leveraging local inference tools such as LM Studio, which enabled efficient experimentation on consumer hardware. Nevertheless, inference latency, memory ceilings, and model size constraints limited the scope of evaluation.

Second, the dynamically generated open-schema ontology led to significant schema fragmentation. Across only 15 documents, the system produced 320 distinct node labels and 423 relationship types. The lack of normalization reduces cross-document comparability and complicates higher-level analytical tasks without additional ontology consolidation or post-processing.

Third, relationship coverage for canonical financial patterns was limited. Although the system generated a large number of fine-grained, context-specific relationship types, only a small percentage of expected standardized financial relations were consistently represented. This suggests that the model often favored generating specialized semantic edges over reusing stable, canonical relationship patterns, limiting analytical generalizability.

Fourth, document-level graph connectivity remained sparse, with an average total node degree of 1.64. This constrains multi-hop reasoning and broader structural analysis across the corpus without further graph enrichment or cross-document entity linking.

Fifth, the hosted API is subject to GPU memory constraints imposed by the shared Modal deployment. The current instance runs on an A10G GPU (22 GB VRAM), which is sufficient for documents up to approximately 50 pages but will exceed available memory for longer documents such as large SEC filings or extended research reports. Users who require processing of larger documents can deploy their own API instance via Modal by cloning the

repository and running `modal deploy src/api/modal_app.py`, which allows them to select a larger GPU tier (e.g., A100 or H100) through Modal’s infrastructure.

Finally, several limitations are attributable to model capacity. Due to local compute constraints, experimentation relied primarily on smaller open-source models. These models exhibited variability when handling forward-looking financial language, long-range dependencies, and cross-sentence coreference. Access to larger state-of-the-art models via hosted APIs could plausibly mitigate some of these issues [Ganesh et al. \(2024\)](#). More capable models generally demonstrate stronger semantic consistency, improved entity resolution, and more stable relation naming behavior, which may reduce ontology fragmentation and improve standardized relationship coverage. However, such approaches introduce trade-offs in cost, latency, and data governance considerations that must be carefully evaluated [Desai et al. \(2024\)](#).

5 Conclusion

Market Foundry demonstrates that schema-guided LLM extraction, combined with dynamic document classification and graph-based storage, is a viable and scalable approach for transforming unstructured financial text into structured knowledge graphs. By processing heterogeneous document types—research reports, news articles, earnings call transcripts, and regulatory filings—into unified Neo4j graphs, the system establishes a foundation for explanation-aware financial intelligence that moves beyond isolated metrics toward connected, causal representations of market events.

The deployment of a hosted REST API marks a meaningful step toward making financial knowledge graph construction accessible beyond research environments. API runtime testing confirmed that the pipeline processes documents ranging from 1 to 36 pages in 5 to 26 minutes with consistent startup overhead, enabling practical integration into real-world analytical workflows without requiring local GPU infrastructure. These results position Market Foundry as a practical bridge between raw financial documents and the structured intelligence layer that downstream analytics, RAG systems, and causal inference workflows require.

Challenges remain in ontology normalization, relation standardization, and robustness on long-form documents, and the hosted API is currently constrained to documents of approximately 50 pages or fewer due to GPU memory limitations. Addressing these gaps, through schema constraint enforcement, document chunking, and task decomposition strategies such as those proposed in DLISC [Wen et al. \(2025\)](#), represents the natural next step toward a production-grade financial knowledge graph system. As financial workflows increasingly demand structured, cross-document reasoning at scale, Market Foundry offers a principled and extensible architecture for building the intelligence infrastructure that modern financial analysis requires.

References

- Ali, Mohamed, Zaki Taha, and Mohamed Morsey. 2026. "Ontology-grounded knowledge graphs for mitigating hallucinations in large language models for clinical question answering." *Journal of biomedical informatics* 175, p. 104993. [\[Link\]](#)
- Arun, Abhinav, Fabrizio Dimino, Tejas Agarwal, Bhaskarjit Sarmah, and Stefano Pasquali. 2025. "FinReflectKG: Agentic Construction and Evaluation of Financial Knowledge Graphs." [\[Link\]](#)
- Bai, Fan, Junmo Kang, Gabriel Stanovsky, Dayne Freitag, Mark Dredze, and Alan Ritter. 2024. "Schema-Driven Information Extraction from Heterogeneous Tables." [\[Link\]](#)
- Belem, Catarina G., Pouya Pezeshkpour, Hayate Iso, Seiji Maekawa, Nikita Bhutani, and Estevam Hruschka. 2025. "From Single to Multi: How LLMs Hallucinate in Multi-Document Summarization." [\[Link\]](#)
- Desai, Akshar Prabhu, Tejasvi Ravi, Mohammad Luqman, Ganesh Mallya, Nithya Kota, and Pranjul Yadav. 2024. "Opportunities and Challenges of Generative-AI in Finance." In *2024 IEEE International Conference on Big Data (BigData)*. IEEE. [\[Link\]](#)
- Ganesh, Sumitra, Leo Ardon, Daniel Borrajo, Deepeka Garg, Udari Sehwa, Annapoorani Narayanan, Giuseppe Canonaco, and Manuela Veloso. 2024. "Generative AI Agents for Knowledge Work Augmentation in Finance." *Annual Review of Control, Robotics, and Autonomous Systems* 8. [\[Link\]](#)
- Huang, Yanlong, Wenxin Tai, Fan Zhou, Qiang Gao, and Ting Zhong. 2025. "Extracting key insights from earnings call transcript via information-theoretic contrastive learning." *Information Processing Management* 62, p. 103998. [\[Link\]](#)
- Luo, Yujie, Xiangyuan Ru, Kangwei Liu, Lin Yuan, Mengshu Sun, Ningyu Zhang, Lei Liang, Zhiqiang Zhang, Jun Zhou, Lanning Wei, Da Zheng, Haofen Wang, and Huijun Chen. 2025. "OneKE: A Dockerized Schema-Guided LLM Agent-based Knowledge Extraction System." In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*. Sydney, NSW, Australia ACM. [\[Link\]](#)
- Shah, Agam, Liqin Ye, Sebastian Jaskowski, Wei Xu, and Sudheer Chava. 2025. "Beyond the Reported Cutoff: Where Large Language Models Fall Short on Financial Knowledge." [\[Link\]](#)
- Shrimal, Anubhav, Aryan Jain, Soumyajit Chowdhury, and Promod Yenigalla. 2025. "PARSE: LLM Driven Schema Optimization for Reliable Entity Extraction." [\[Link\]](#)
- Wen, Zhihao, Sheng Liang, Yaxiong Wu, Yongyue Zhang, and Yong Liu. 2025. "Effective and Efficient Schema-aware Information Extraction Using On-Device Large Language Models." [\[Link\]](#)
- Yu, Hong Qing, and Frank McQuade. 2025. "RAG-KG-IL: A Multi-Agent Hybrid Framework for Reducing Hallucinations and Enhancing LLM Reasoning through RAG and Incremental Knowledge Graph Learning Integration." [\[Link\]](#)

Appendices

A Additional Figures

Table A 1: Additional Knowledge Graph Visualizations

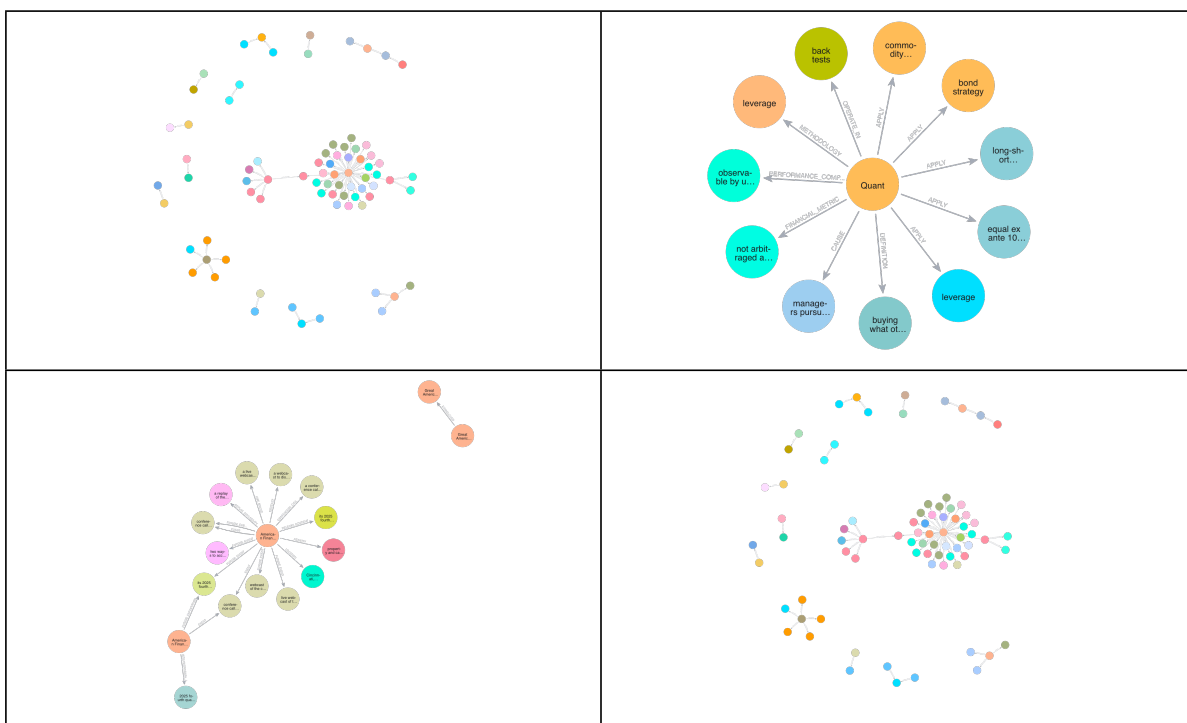


Table A 2: The entire knowledge base from 15 extracted documents.

